

## Упражнение 10 – Проектиране, създаване, оптимизиране и рефакторизиране на софтуер

**Цел на упражнението:** Основната цел на десетото упражнение е запознаване на студентите с общи похвати в софтуерното производство.

Разглеждат се подходи за проектиране на софтуер, например създаване на библиотеки, структуриране на код по удобен за модификации начин.

По отношение на създаването на софтуер се разглеждат различни конвенции при софтуерната разработка – именуване, кодиране, визия на кода.

Разглежда се необходимостта от оптимизиране с цел спазванията на клиентските изисквания за производителност.

В упражнението се акцентира върху рефакторизирането (рефакторинга) като метод за опростяване на софтуера и в частност на неговата структура.

**Ключови думи:** software design, conventions, optimizations, refactoring.

### Съдържание

|                                                                |    |
|----------------------------------------------------------------|----|
| 1. Проектиране на софтуера .....                               | 1  |
| 1.1. Нива на софтуерно проектиране[1] .....                    | 2  |
| 1.2. Разширяемост на системата .....                           | 2  |
| 1.3. Разделяне по модули (модуларизация, modularization) ..... | 2  |
| 1.4. Начини за разделяне на функции/класове по модули[1] ..... | 3  |
| 1.5. Връзки между модулите .....                               | 3  |
| 1.6. Шаблони .....                                             | 4  |
| 1.7. Обобщение .....                                           | 5  |
| 2. Конвенции .....                                             | 6  |
| 2.1. Езиково-независими конвенции .....                        | 6  |
| 2.2. Езиково-зависими конвенции .....                          | 7  |
| 3. Оптимизиране .....                                          | 7  |
| 4. Рефакторизиране (refactoring, рефакторинг) .....            | 8  |
| 4.1. Причини за рефакторизиране .....                          | 8  |
| 4.2. Рефакторизиране на данни .....                            | 9  |
| 5. Задачи за изпълнение .....                                  | 9  |
| Задача 1 .....                                                 | 9  |
| Задача 2 .....                                                 | 9  |
| Задача 3 .....                                                 | 9  |
| Задача 4 .....                                                 | 9  |
| Задача 5 .....                                                 | 10 |

### 1. Проектиране на софтуера

Проектирането на структурата на софтуера не винаги е задача на project manager-а, но е много полезно ако той може да го направи. Ако даден проект е твърде голям и трябва да се реализира от няколко екипа, то project manager-ът е човекът, който има поглед над целия проект. На базата на знанието за потребителските изисквания и разпределянето на задачите между различните екипи, project manager-ът трябва да проектира софтуера така, че:

- ⑩ различните екипи да имат относително автономни задачи (за да не си пречат);
- ⑩ да няма повторение на код;

- ⑩ екипите по проекта да спазват общи фирмени конвенции (по-подробно разгледано в т. 6 Конвенции, стр. 6;
- ⑩ да се преизползва фирмен софтуер, вместо да се създава отново;
- ⑩ системата да бъде лесна за модификации и разширяване;
- ⑩ и др.

### **1.1. Нива на софтуерно проектиране[1]**

Проектирането на софтуера може да се направи на няколко нива:

- ⑩ проектиране на архитектурата на софтуера – това е процес на описание на софтуера на най-високото ниво на абстракция. Софтуерът се описва като система с множество компоненти, които си взаимодействат с цел постигане на клиентските изисквания;
- ⑩ проектиране на високо ниво – това е представяне на софтуера на малко по-ниско ниво на абстракция. Тук акцент пада върху разпределянето на системата в пакети, подсистеми, модули и т.н;
- ⑩ детайлен дизайн – този етап се свързва с имплементирането на това какво се вижда от системата и отделните подсистеми. Тук се планира структурата на всеки модул и интерфейсите за комуникация между модулите.

### **1.2. Разширяемост на системата**

Един от факторите, които проектиращият трябва да съблюдава, е разширяемостта на системата. Проектирането трябва да е такова, че да позволява лесна модификация и добавяне на нови функционалности. Общият принцип за осигуряване на разширяемостта на системата е използването на подсистеми. Така се осъществява разделяне на един сложен проблем на по-малки и по-прости подпроблеми.

Разделянето става по принципа:

- ⑩ Софтуерна система
- ⑩ Подсистеми и именовани пространства - Съществуват някои стандартни подсистеми, които често се използват при разработването на софтуер:
  - Бизнес логика;
  - Потребителски интерфейс;
  - Достъп до БД;
  - Системни зависимости.
- ⑩ Разделяне на класове и интерфейси;
- ⑩ Разделяне на методи;
- ⑩ Вътрешно проектиране на методите.

### **1.3. Разделяне по модули (модуларизация, modularization)**

Разделянето по модули е техника за разделяне на системата в множество независими модули, всеки от които има определена конкретна задача. Може да се каже, че разделянето по модули представлява на практика „Разделяй и владей“ (Divide and conquer). Модули се оформят като базова конструкция на софтуера на проекта. Някои от съображенията при разделянето по модули са[1]:

- ⑩ получаване на независими фрагменти код, които могат да се изпълняват или компилират поотделно;
- ⑩ по-малките компоненти са по-лесни за поддръжка;

- ⑩ програмата може да се раздели на принципа на различните функционалности;
- ⑩ чрез модулите може да се постигне желаното ниво на абстракция;
- ⑩ отделните компоненти могат да се преизползват;
- ⑩ и др.

#### **1.4. Начини за разделяне на функции/класове по модули[1]**

Разглеждат се следните принципи, на базата на които функциите/класовете могат да се разпределят по модули:

- ⑩ на случаен принцип – не се препоръчва, тъй като не се спазва никаква организация. Подобен проект се управлява и мащабира трудно;
- ⑩ логически принцип – функциите/класовете се подреждат логически, т.е. функции и класове, които са логически свързани, се слагат на едно място;
- ⑩ времеви принцип – функциите/класовете се подреждат според това дали се изпълняват в близък времеви момент;
- ⑩ процедурен принцип – функциите/класовете се подреждат последователно, по реда на тяхното изпълняване;
- ⑩ комуникационен принцип – функции/класове, които използват едни и същи данни, се подреждат в един модул;
- ⑩ последователен принцип – функциите/класовете се групират, защото изходът от едната функция/клас представлява вход за друга функция/клас от същия модул;
- ⑩ функционален принцип – функциите/класовете от един модул се използват, за да осигурят конкретна функционалност на софтуера.

#### **1.5. Връзки между модулите**

Връзки между модули:

- ⑩ връзка по отношение на съдържанието – модул може директно да достъпва съдържание на друг модул;
- ⑩ споделена връзка – модул има възможност да чете и пише съдържанието на глобални данни (разположени в друг модул);
- ⑩ даннова връзка – модулите си взаимодействат като обменят помежду си данни.

Свързаност между модулите:

- ⑩ Параметрична свързаност - Два модула са свързани параметрично, ако единственият начин за предаване на информация между тях е, чрез множество от параметри, като всички те трябва да са от примитивен тип;
- ⑩ Обектна свързаност - Един модул е обектно свързан към даден обект, ако в себе си създава този обект;
- ⑩ Обектно-параметрична свързаност. Два модула са обектно-параметрично свързани, ако обект 1 иска от обект 2 да получи обект 3. Тази свързаност е по-широка отколкото ако обект 1 иска от обект 2 да му подаде данни само от примитивен тип.
- ⑩ Семантична свързаност. Това е най-неприемливият вид свързаност и тя се получава когато един модул не използва синтактични елементи от друг модул, а определен вид семантична информация за начина на неговата работа. Пример: Функция в даден модул прави анализ от вида: „Ако ме извикат от модул X, правя това, а ако ме извикат от модул „Y”, правя друго.

## 1.6. Шаблони

Практиката е довела до формирането на множество структурни шаблони (structure design patterns) - Шаблоните за проектиране представляват стандартизирани опростени обектни структури, които се използват за решаване на често срещани проблеми в практиката и имат установени имена. Голяма част от проблемите, при проектиране на определена част от системата, са вече срещани от други програмисти в миналото и са добре изследвани и изучени. Познаването им позволява да се изградят добре структурирани системи, подсистеми и модули. Някои от най-често използваните шаблони за проектиране включват: **Adapter, Bridge, Decorator, Facade, Factory Method, Observer, Singleton, Strategy, Template Method** и др. Някои от предимствата при използването им са следните:

- ⑩ Намалават сложността на системата, чрез абстракции. Използването на стандартно наименование за дадена обектна структура улеснява предаването на информация между участниците в екипа, и премахва необходимостта от разглеждане на всеки ред от програмния код, за да се разбере смисъла на дадена програмна структура;
- ⑩ Намалават вероятността за възникване на грешки. Детайлите свързани с някои проблеми при проектирането се изясняват едва след съставяне на програмната реализация. Тъй като шаблоните са стандартизирани начини за решаване на често срещани проблеми, те съдържат в себе си акумулиран с времето опит от решаването на такива проблеми. Използването на шаблони за проектиране може концептуално да се оприличи на използване на библиотека с функции, вместо да се написва собствена такава. Голяма част от проблемите при проектиране са подобни на вече срещани проблеми и в този смисъл е добре да се използват известните решения, вместо да се създават нови такива.
- ⑩ Всеки който е запознат с шаблоните, винаги може да обмисли всеки един от тях и да реши дали има такива, които съответстват на неговия проблем и могат да бъдат използвани;
- ⑩ Служат като предложения. Разглеждането на множество от известни алтернативни решения е много по-лесно отколкото създаването на добро собствено решение, различно от всички останали. Освен това програмният код на известните решения обикновено е по-лесен за разбиране, в сравнение със специфичните собствени решения;
- ⑩ Шаблоните опростяват комуникацията между участниците в екипа, като дават основа за диалог на по-високо ниво. Освен предимствата на шаблоните при управление на сложността, съществуват и предимства свързани с подобряване на комуникацията при проектиране, давайки възможност на разработчиците да разсъждават на по-високо ниво на абстракция. Така се постига обмен на голямо количество информация с малко на брой думи.

Някои от най-често използваните шаблони са показани в таблица 11.1.

Таблица 1 Често използвани шаблони за проектиране

| Шаблон           | Описание                                                                                                                                                           |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Abstract Factory | Използва се за създаване на група от обекти, чрез задаване на типа на групата                                                                                      |
| Adapter          | Конвертира интерфейс на клас към друг интерфейс                                                                                                                    |
| Bridge           | Създава интерфейс и имплементация по такъв начин, че всеки от тях може да се променя независимо от другия                                                          |
| Composite        | Обекти съдържащ допълнителни обекти, от неговия тип, като клиентът може да взаимодейства с обекта от най-високо ниво без да се интересува от допълнителните обекти |
| Decorator        | Дава възможност за динамично добавяне на функционалности към обект, без да се създават подкласове                                                                  |
| Facade           | Осигурява консистентен интерфейс за неконсистентен програмен код                                                                                                   |

|                 |                                                                                                                                                                |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Factory Method  | Създава обекти от класове наследници на определен базов клас, като за целта се използва само определен метод                                                   |
| Iterator        | Сървърен обект, който последователно дава достъп до всеки елемент в дадено множество                                                                           |
| Observer        | Осъществява синхронизация между множество от обекти, чрез трети обект, който изпраща съобщения към обектите от множеството при промяна в някои от членовете им |
| Singleton       | Осигурява глобален достъп до клас, който може да има само един обект                                                                                           |
| Strategy        | Дефинира множеството от динамично променящи се алгоритми                                                                                                       |
| Template Method | Дефинира структура на алгоритъм, някой от детайлите на който се реализират в класовете наследници                                                              |

## 1.7. Обобщение

Проектирането на софтуер е отговорна задача и има смисъл да се прави от project manager или team leader, който има поглед над целия проект. Обикновено структурирането на проект е операция, която е частна за самия проект, но в общия случай в рамките на една фирма е хубаво проектите да се структурират по един и същ общ начин.

Най-общо проектирането се разделя на:

- ⑩ структурно проектиране – проектиране на разполагането на софтуера по модули и пакети;
- ⑩ алгоритмично проектиране – проектиране на отделните алгоритми, реализирани от софтуера;
- ⑩ данново проектиране – проектиране на данните, с които софтуерът работи – напр. Структура на БД.

Препоръки:

- ⑩ Класовете трябва да съответстват на обекти от реалния свят - При дефиниране на класовете се изпълняват следните стъпки.
  - Определяне на обектите и техните атрибути (методи и данни).
  - Определяне на действията, които ще се извършват върху обекта.
  - Определяне на действията извършвани от обекта.
  - Определяне на public и private членовете на обекта.
  - Дефиниране на публичния интерфейс на обекта.
- ⑩ Изграждане на абстракция - Абстракциите се реализират, чрез базовите класове и интерфейсите, като тяхното предназначение е да осигурят възможност за работа с концепции, при което детайлите се игнорират. Базовите класове съдържат общите атрибути на специфичните класове, а интерфейсите дават възможност за фокусиране върху интерфейсите на специфичните класове, като действията в методите им не се определят. Добре дефинираните абстракции улесняват проектирането на останалите нива на системата. Например, добре проектираните подсистеми дават възможност за съставяне и на подходящо множество от интерфейси на по-долно ниво.
- ⑩ Капсулиране - Капсулирането дава възможност за ограничаване на видимостта на детайлите относно реализацията;
- ⑩ Наследяване с цел опростяване на системата - В някои случаи в системата съществуват обекти, които имат много общи характеристики. В този случай те трябва да се обобщят и обединят в базов клас, а специфичните детайли на различаващите се обекти да се обособят в самостоятелен клас. Наследяването опростява програмирането, тъй като основните характеристики се реализират в обобщаващи методи, а специфичните в специфични методи;
- ⑩ Когато става въпрос за софтуер, който комуникира с външни страни, то е добра идея комуникацията да бъде отделена в слой, който е различен от слоевете за данни и изобразяване.

## 2. Конвенции

Конвенциите са правила, наложени в рамките на дадена общност, които имат препоръчителен характер (клонящ към задължителен). Общността може да се формира от разработчиците в дадена фирма/екип или от разработчици, използващи определена технология.

Основен плюс на спазването на конвенции е разбираемостта, която се подобрява. Когато програмистите в дадена общност използват едни и същи конвенции, за тях е по-лесно да се разбират, както и да четат чужд код помежду си.

В някои случаи конвенциите се използват, за да компенсират недостатъците на някои програмни езици. Например в името на променливата може да се сложи буквичка, която показва типа на променливата, за програмните езици, които не са типизирани.

### 2.1. Езиково-независими конвенции

Някои езиково-независими конвенции:

- ⑩ Променливите и методите трябва да се именуват по различни начини (започваща главна или малка буква, използван разделител и др.);
- ⑩ Класовете и обектите трябва да се именуват по различен начин (започваща главна или малка буква, използван разделител и др.);
- ⑩ Глобалните променливи трябва да са различни по името си;
- ⑩ Член-променливите на клас трябва да са идентифицируеми като такива;
- ⑩ Дефиницията на клас трябва да може да се различава от останалите имена;
- ⑩ Именованите константи трябва да са различни от останалите елементи. Един възможен начин е имената им да са с изцяло главни букви;
- ⑩ Изброимите типове трябва да са различни от останалите елементи;
- ⑩ Входните и изходните параметри на даден метод трябва да могат да се разпознават;
- ⑩ Трябва да се използва единно форматиране за имената на програмните конструкции. Всяка дума в имената например може да започва с главна буква;
- ⑩ Имена на методи:
  - Името на метода трябва да описва това което реално се извършва в него;
  - Трябва да се избягват глаголи с неясно значение;
  - Не трябва да има методи с еднакви имена и различаващ се номер в името. Например методи с имена OutputUser, OutputUser1, OutputUser2 и т.н. ;
  - Имената на методите трябва да са толкова дълги, колкото е необходимо.
- ⑩ Самодокументиращ се програмен код – Това не е документиране чрез външна документация или коментари, а документиране чрез самия стил на програмиране. Всяка документация остава неактуална след промени в кода, но кодът сам по себе си винаги е актуален. Самодокументирането включва добро структуриране на програмата, използване на ясни и разбираеми подходи за писане, добри имена на променливи и методи, използване на именувани константи вместо числови стойности, правилно подреждане, минимизиране на сложността на данните структури и др.
- ⑩ Цикли:
  - В цикли организирани с **while**, **do-while**, инкрементирането, декрементирането и модификацията на контролните променливи, трябва да се извършват само в началото или в края на тялото на цикъла;
  - Инициализациите свързани с цикъла трябва да се записват максимално близо до цикъла;

- Влагането на цикли трябва да се ограничи най-много до три нива. При по-големи нива на влагане, възможностите им за управление силно се влошава. Ако все пак се налага да се правят влагания на повече нива, те трябва максимално да се опростят, например чрез реализация на действията в отделни методи;
- Трябва да се избягва използването на последната стойност на брояча на цикъл в програмен код след цикъла, тъй като това предразполага към възникване на грешки;
- В тялото на цикъла не трябва да се модифицира стойността на брояча. Някои програмисти използват тази техника за предизвикване на край на цикъла, което не е добра практика.

## 2.2. Езиково-зависими конвенции

Съществуват различни езиково-специфични конвенции, за които има достатъчно източници в Интернет. В тази точка ще се разгледат насоките в тези конвенции, които е хубаво да се спазват:

- ⑩ именуване на методи;
- ⑩ именуване на класове;
- ⑩ именуване на функции;
- ⑩ именуване на променливи от примитивен тип;
- ⑩ именуване на обекти;
- ⑩ именуване на модули;
- ⑩ именуване на пакети/именувани пространства;
- ⑩ място на отварящата скоба;
- ⑩ именуване на итератори на цикли;
- ⑩ именуване на константи;
- ⑩ именуване на глобални променливи.

Когато започва изучаването на даден програмен език, учещият несъзнателно се запознава с конвенциите за съответния език, най-малкото защото всички публични източници се стараят да ги спазват.

Използването на такива конвенции е задължително, ако ще се отправя запитване към общността от разработчици, използващи езика, като се прикача програмен код.

## 3. Оптимизиране

В някои случаи е важно да се пише оптимизиран код. На практика е трудно още по време на разработката на кода, кодът да се пише така, че да бъде оптимизиран и затова стъпката по оптимизация следва стъпката по разработка. Оптимизирането до голяма степен върви свързано с рефакторирането (разгледано подробно в т. 8 Рефакториране (refactoring, рефакторинг), стр. 8).

Най-общата дефиниция на думата „оптимизиране“ е: повишаване производителността [2]; Избирам най-добрия вариант от възможните, за да направя оптимален избор.

Оптимизациите в контекста на софтуерното производство могат да се състоят в реструктуриране на кода (рефакториране), подобряване на използваните алгоритми чрез по-ефективни такива, по-добро управление на ресурсите (памет; изчислителни ресурси).

Случаи, в които оптимизирането може да бъде наложително:

- ⑩ бързодействието е нефункционално изискване към софтуера – например дадена заявка да се изпълнява за по-малко от 1 секунда;
- ⑩ разполага се с ограничени изчислителни ресурси (оперативна памет, външна памет) – програмиране на контролер.

Извън отбелязаните случаи, оптимизирането не е задължителна операция, но е желателно при възможност да се отдели време за него, за да се щадят изчислителните ресурси и търпението на клиента.

При оптимизиране трябва да се вземат предвид по-бавните операции:

- ⑩ установяване на връзка към БД;
- ⑩ комуникация с трета страна по определен протокол (напр. Http);
- ⑩ дискови операции – четене и запис;
- ⑩ създаване на обекти с множество големи по обем член-данни; God обекти и обекти със self-reference-и.
- ⑩ рекурсия;
- ⑩ цикли с множество итерации;

## 4. Рефакториране (refactoring, рефакторинг)

Рефакторирането се дефинира като "промяна във вътрешната структура на софтуера, която без да променя възможностите му, го прави по-лесен за разбиране и модифициране".

### 4.1. Причини за рефакториране

- ⑩ Повторение на програмния код;
- ⑩ Наличие на големи методи с голяма и сложна логика;
- ⑩ Дълги или дълбоко вложени цикли;
- ⑩ Даден клас изпълнява множество функции (В даден клас са съсредоточени множество от несвързани действия);
- ⑩ Списъкът с параметри на даден метод е твърде дълъг;
- ⑩ Когато се извършва промяна се налага паралелна модификация на няколко класа;
- ⑩ Свързани елементи, които се използват заедно, не са включени в общ клас;
- ⑩ Даден метод използва повече функционалност на друг клас, отколкото на своя собствен;
- ⑩ Неправилно използване на примитивни даннови типове;
- ⑩ Даден клас няма особено голямо значение;
- ⑩ Един клас е силно свързан с друг клас;
- ⑩ Лошо именован метод;
- ⑩ Публични член променливи на клас;
- ⑩ Клас наследник използва много малка част от методите на базовия клас;
- ⑩ Използване на глобални променливи;
- ⑩ Програмата съдържа неизползваем код, който се предполага, че ще се използва след време.

### 4.2. Рефакториране на данни

- ⑩ Използване на именовани константи вместо реалните им стойности;
- ⑩ Променливите трябва да имат ясни и информативни имена;
- ⑩ Използване на метод вместо израз;
- ⑩ Използване на междинни променливи за резултат от израз;
- ⑩ Записване на израз вместо променлива;
- ⑩ Трябва да се избягва използване на една променлива за повече от една цел;



- ⑩ Параметрите на методите не трябва да се използват като локални променливи в него.

## 5. Задачи за изпълнение

### Задача 1

Кои са местата в дадена информационна система, които подлежат най-често на промени, и чието проектиране трябва да бъде огледано внимателно за възможности за разширяемост?

### Задача 2

В кои случаи трябва да се използват конвенции и в кои случаи е препоръчително това да се прави? Има ли случаи, в които спазването на конвенции пречи на софтуерната разработка?

### Задача 3

За всяка от причините за рефакторизиране, описани в т. 8 Рефакторизиране (refactoring, рефакторинг), стр. 8 Определете:

- ⑩ Защо е нужно рефакторизирането, т.е. защо е хубаво да се направи промяната?
- ⑩ Как може да се направи промяната и колко места ще засегне?

### Задача 4

Продължение на задачата от упражнение 5 и упражнение 8. В упражнение 5 трябваше посредством избрана система за управление да се въведат стъпки и задачи по провеждането на първоначално проучване и планиране на конкретна информационна система. Предложените задачи бяха:

- ⑩ информационна система за регистратура в медицински център;
- ⑩ информационна система за следене на градския транспорт;
- ⑩ информационна система – регистър на студенти в университет;
- ⑩ складова информационна система за аптека;
- ⑩ складова информационна система за супермаркет;
- ⑩ складова информационна система за железария.

В упражнение 8 трябваше да се изготви проектен план за избраната задача.

Текуща задача:

Направете схеми, които показват структурата на изборения от вас софтуер. Проектирайте структурата, данните и алгоритмите (на най-високо ниво).

### Задача 5

Разгледайте предложените примери за рефакторизиране. Примерите са взети от <https://lostechies.com/wp-content/uploads/2011/03/31DaysRefactoring.pdf> [3]

....TODO: Да се опише как точно да се стигне до примерите на виртуалната машина!...

## Използвана литература

1: tutorialspoint, Software Design Basics, ,  
[https://www.tutorialspoint.com/software\\_engineering/software\\_design\\_basics.htm](https://www.tutorialspoint.com/software_engineering/software_design_basics.htm)